Preventive Maintenance Web App — Solution Document

Version: 2.0

Date: 2026-08-22

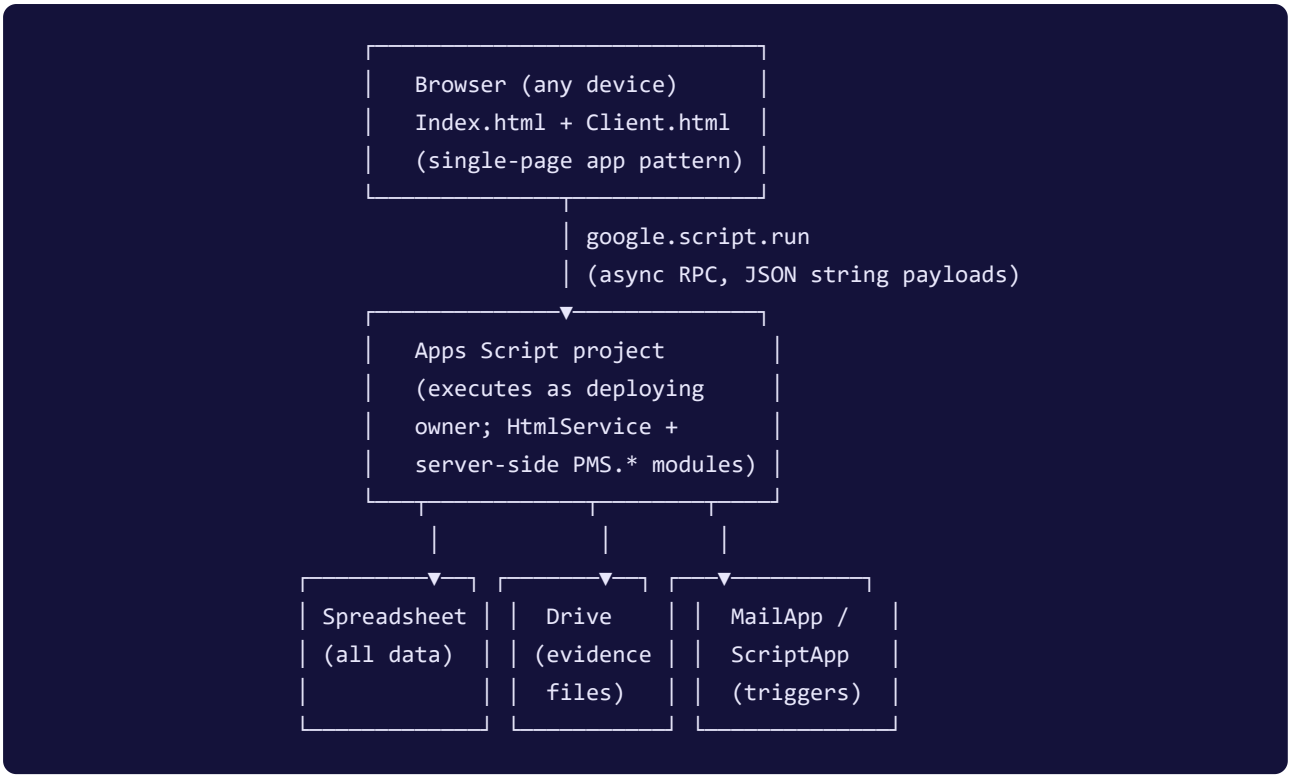
Audience: Engineers, technical reviewers, and anyone who needs to understand how the system is built rather than what it does. For behavioral requirements, see `PRD.md`. For business justification, see `BRD.md`. For task-by-task instructions, see `USER_GUIDE.md`.

1. Purpose and scope

This document describes the technical architecture of the YDC Preventive Maintenance System (PMS): how it is built, what it is built from, how its pieces communicate, how data is modeled and protected, and how it is deployed and verified. It complements the PRD rather than repeating it — where the PRD says a completed record can never be edited again, this document explains the mechanism that enforces that.

2. Architectural overview

The application is a single Google Apps Script project, deployed as a web app, with a Google Sheets spreadsheet as its sole datastore and Google Drive as its evidence store. There is no separate database, no separate backend server, and no build step in production: Apps Script serves HTML directly to the browser and executes server logic in the same script project, on Google's infrastructure, under Google's execution model.



Key architectural decisions and their reasons:

- **Execute as the deploying owner (`executeAs: USER_DEPLOYING`).** A technician never needs direct edit access to the spreadsheet or Drive folders. Every read and write goes through server code that enforces authorization itself; the browser holds no credentials of its own beyond an authenticated Google session.
- **One spreadsheet, many sheets, no separate database.** The organization already lived in a spreadsheet before this project existed; keeping it there means no migration, no second system of record, and lets an administrator still open the workbook directly for anything the app doesn't cover.
- **No client-side framework.** The browser code is vanilla JavaScript inside an HTML Service template, using `fetch`-style promises wrapping `google.script.run`. This is a deliberate constraint of the platform (Apps Script's `HtmlService` cannot serve arbitrary static assets or run a bundler), not a stylistic choice, and the codebase is organized to keep that manageable at its current size (see §5).

3. Technology stack

Layer	Technology
Server runtime	Google Apps Script (V8 runtime, ES5-compatible authoring style used throughout for broad compatibility)
Server-to-browser transport	`google.script.run` (Apps Script's RPC bridge); payloads are JSON-stringified server-side and parsed client-side, bypassing the structured serializer (see §5.3)
Data storage	Google Sheets (`SpreadsheetApp` service), one workbook, many sheets
File storage	Google Drive (`DriveApp` service), two fixed folders for Infrastructure evidence
Email	`MailApp` service, sent by the deploying account
Scheduling	`ScriptApp` time-based triggers (one recurring daily trigger; one-shot self-rescheduling triggers for bounded background work)
Browser UI	Server-rendered HTML partials (`HtmlService.createHtmlOutputFromFile`), vanilla JS, hand-written CSS (no framework, no bundler, no npm dependency in the shipped app)
Local development / deployment tooling	`clasp` (Command Line Apps Script Projects) for pushing source and creating versioned deployments
Automated testing	A standalone Node.js test harness (not part of the shipped app) that loads the real `WebApp/*.js` files into a `vm` context against hand-written stubs of every Apps Script service used (see §11)

4. Deployment topology

A single Apps Script project backs a single production **web app deployment**, addressed by a fixed deployment ID and served at a stable `/exec` URL. `clasp push` uploads the current state of every file in `WebApp/` to the project; a separate `clasp deploy -i <deploymentId>` step then points that deployment's live URL at the version just pushed, with a short description recorded per deployment (visible via `clasp deployments`). Pushing alone never changes what the live URL serves — a deploy step is always required to actually ship a change, which keeps "code is on the project" and "code is live for users" as two distinct, deliberate steps.

The manifest (`appsscript.json`) declares:

- `executeAs: USER_DEPLOYING` and `access` restricted to the YDC Google Workspace domain.
- OAuth scopes: `userinfo.email` (so `Session.getActiveUser()` is reliably populated for a same-domain visitor, per §6.1) and the full `drive` scope (needed because the app writes to two fixed, pre-existing folders whose IDs are known in advance rather than created per-user).
- A time zone of `Asia/Manila` , matching every date/time computation in the server code.

5. Application structure

5.1 File-loading model and its consequences

Apps Script evaluates every `.js` file in a project **alphabetically** at load time, before any function is called. Every server module in this codebase follows the same shape to work safely under that constraint:

```
var PMS = PMS || {};
PMS.ModuleName = (function () {
  // private helpers
  return { /* public functions */ };
})();
```

Because load order is alphabetical and fixed, a module cannot reference another module's exports, or even `PMS.CONFIG` / `PMS.Util`, at the top level of its own IIFE — only inside a function body, which doesn't execute until some caller invokes it, by which point every file has finished loading. `PmsNotify.js` and `PmsTickets.js` both carry explicit comments about this, since they are early in alphabetical order and reference config/utility functions defined in later-loading files.

5.2 Server module inventory

File	Responsibility
`PmsConfig.js`	All configuration as one frozen object: spreadsheet ID, sheet names, cycle definitions, checklist/peripheral/asset-type schemas, evidence folder IDs and limits, notification settings, and the various administrator-tool thresholds and timeouts. Nothing here is a secret; secrets live in Script Properties instead (§7.4).
`PmsUtils.js`	Shared low-level helpers: date parsing/formatting in `Asia/Manila`, text cleaning, email normalization, checklist-value parsing, the `fail(message, code)` error convention used everywhere, cache helpers, and the completion-progress-bar text builder (`progressText`).
`PmsValidation.js`	The shared payload funnel every questionnaire save goes through — checklist normalization (shared shape, different item lists per section), peripheral normalization, Infra asset-type/evidence handling, and assessment validation.
`PmsAuth.js`	Identity resolution, domain/roster enforcement, registration, the admin/asset-manager/technician permission checks used by every other module, and the administrator functions behind the Manage Users screen.
`PmsUsers.js`	The `PMS Users` roster sheet: schema, self-healing header migration, profile read/write, and the computed role/administrator-status logic described in §7.2.
`PmsAssets.js`	Bounded, cached reads of each section's asset master; `INPROD` filtering; per-cycle eligibility; the server-side re-validation every save and evidence upload independently repeats.
`PmsAssetAdmin.js`	The Manage Assets screen's backing logic: list (open to any registered user), create/update/bulk-CSV (Asset Manager or administrator only).
`PmsRecords.js`	The core questionnaire save/complete pipeline: idempotency, duplicate-draft detection and adoption, the permanent-lock check on a completed record, legacy-seed record construction, and the dashboard/archive/duplicate-detection read paths that aggregate both section sheets.
`PmsEvidence.js`	Infrastructure evidence upload, validation, content-sniffing, descriptor signing/verification, and the re-verification path invoked at completion time.
`PmsTracker.js`	The controlled writes to the two tracker sheets' T1–T3 status/remarks cells — the only normal-operation writes to pre-existing workbook cells — including the repair-aware status computation that reads ticket state.
`PmsTickets.js`	The findings-ticket module: creation, status transitions, the append-only change log, and the ticket-status-to-tracker-cell mapping.
`PmsMetrics.js`	Dashboard aggregation: eligible/completed/deferred/pending counts, compliance percentage, findings/follow-up counts, completion-by-location, all de-duplicated by asset+year+cycle.
`PmsNotify.js`	All outbound email: the shared send/template infrastructure, the three notification types (completion, ticket events, cycle-deadline reminder), and the daily trigger's

File	Responsibility
	installation.
`PmsLegacyImport.js`	The bulk backfill wizard's preview/execute logic, including classification, token-bound confirmation, and chunked execution.
`PmsRollover.js`	Annual rollover: dry run, archive-then-reset execution, audit events, and the self-rescheduling reconciliation trigger for pending records.
`PmsYearPurge.js`	Permanent deletion of one closed year's data, gated by a mandatory backup-download step.
`PmsReset.js`	Full pilot/test-data clearing across every sheet and year, independent of Rollover/Purge.
`PmsWebApp.js`	The web app's entry point (`doGet`), the HTML partial include helper, and every `PMS_api*` function the browser actually calls — each one a thin, auth-checked, try/caught wrapper around the module functions above. Also holds the small number of top-level, non-namespaced functions that must exist for Apps Script's editor Run dropdown or its trigger system to find them by name (see §5.4).

5.3 Client architecture

The browser side is one long-lived single page. `Index.html` is the shell: it declares the SVG icon sprite used everywhere, then includes every other HTML partial in sequence via a small server-side template helper (`PMS_include`), and finally includes `Client.html` last. Because Apps Script's `HtmlService` cannot serve a standalone `.js` file to the browser, all client logic lives inside `Client.html` 's `<script>` block — this is a platform constraint, not a structural choice, and the file is organized into clearly commented sections (bootstrap, dashboard rendering, questionnaire, archive/drafts/tickets/deferred-assets modals, asset/user admin screens, rollover/purge/reset panels, shared combobox/focus-trap/modal utilities) rather than split into multiple files.

State is a single in-memory `state` object, mutated directly and re-rendered by calling the relevant `renderX()` function after each change — there is no virtual DOM or reactive framework; each screen's render function reads from `state` and writes DOM directly via `document.createElement` / `textContent` (never raw HTML interpolation of user data, to avoid XSS).

Server calls go through a `serverCall(functionName, ...args)` wrapper around `google.script.run` , returning a real JavaScript `Promise` . Every `PMS_api*` server function returns a **JSON string**, not a structured object, specifically because `google.script.run` 's built-in structured serializer silently delivers `null` to the success handler once a response is large enough to defeat it — a real risk here, since a single section's asset list plus dashboard metrics can be over a thousand items. Returning a string sidesteps that serializer entirely; the client always JSON-parses the response, and a thrown server error still surfaces through the same `{ok:false, message, code}` shape the rest of the client already expects (`PMS.Util.publicError`).

Preview-server pattern. Because Apps Script normally requires an authenticated deployment to render anything, this codebase also uses a small standalone Node.js HTTP server for local UI verification during development, outside the shipped app: it serves the same `Index.html` /partials with a mock `google.script.run` that proxies calls into the same Node test-stub environment used for automated tests (§11), so a real browser (headless, via Playwright) can exercise the UI against realistic seeded data without a live Google deployment. This tooling lives outside `WebApp/` and is not part of what ships.

5.4 Naming conventions

- Every server function callable from the browser is named `PMS_api<Verb><Noun>` and lives in `PmsWebApp.js`.
- A function that must be directly selectable from the Apps Script editor's Run dropdown, or directly addressable as a trigger handler, **must** be a top-level `function Name() {}` — Apps Script's editor and trigger system cannot select or target a namespaced method like `PMS.Auth.adminSetAssetManager`. Every console-only administrator tool (for example the duplicate-draft cleanup functions, or the cycle-reminder test function) therefore has a thin top-level wrapper in `PmsWebApp.js`, even when the real logic lives inside a `PMS.*` module and is never itself directly callable from the editor.
- A trigger's handler function name conventionally ends in an underscore (`PMS_continueReconciliation_`, `PMS_sendCycleReminder_`), signaling "called by the platform, not by a person," though this is a naming convention only — Apps Script does not enforce anything based on the underscore.

6. Identity and authorization

6.1 Identity resolution

`Session.getActiveUser().getEmail()` is the only source of truth for who is visiting. It returns a populated email only when the visitor is in the same Workspace domain as the deploying owner **and** the manifest declares the `userinfo.email` scope — both conditions are met by this deployment's configuration. `Session.getEffectiveUser()` is never used as an identity fallback: because the app executes as the deploying owner, treating the effective user as the visitor would misattribute every technician's work — and every privilege check — to the owner's own account.

When the live active-user email is temporarily unavailable (a known, occasional Apps Script behavior for a returning visitor), the server may fall back to a hash of `Session.getTemporaryActiveUserKey()`, resolved only against a previously bound, active roster profile. This path is intentionally weak: it can restore an existing technician's session continuity, but it can never register a new user, change a section, or authorize an administrator action, since Google can rotate the temporary key at any time and it is not a durable credential.

6.2 Authorization layering

Every privileged operation is guarded by one of a small number of composable checks in `PmsAuth.js`, each stricter than the last:

- `requireProfile()` — any registered, active user.
- `requireAssetManager()` — a registered user whose roster row has the Asset Manager grant, or an administrator (an administrator always satisfies this without a separate grant).
- `requireAdmin()` — a configured administrator, with a live (not temporary-key) identity.

No client-supplied field is ever trusted for authorization. Section, role, administrator status, and email are all re-derived server-side on every call from the roster and configuration, never taken from the request payload.

6.3 Administrator status as a configuration fact, not a data fact

Administrator status is computed on every read from a static allowlist in `PmsConfig.js` plus an optional Script Property allowlist (`PMS_ADMIN_EMAILS`), never stored as an independently writable roster cell. `PMS.Users`'s row-to-profile function always recomputes both `isAdmin` and the derived `role` label this way; nothing in the write path accepts or persists a client-supplied `isAdmin`. This is what makes it structurally impossible for the in-app Manage Users screen (§9) to grant administrator access, no matter what request it sends — the server function it calls has no code path that writes that field at all.

7. Data model

All data lives in one Google Sheets spreadsheet, addressed by a fixed spreadsheet ID in configuration. Every sheet used by the app is provisioned, and its header row strictly verified, before any write — an unrecognized header shape refuses to use the sheet (`SCHEMA_MISMATCH`) rather than risk writing into the wrong column, and a genuinely blank/uninitialized sheet is auto-formatted in place. Several sheets have self-healing migration for an added column: a newly introduced field heals a blank header cell into an existing live sheet instead of hard-failing every deployment that predates it.

Sheet	Rows represent	Approx. width
`IT-SD PMS`	Service Desk asset master + live tracker columns (D:I = current cycle status/remarks)	9 used columns
`IT-IS PMS`	Infrastructure & Security asset master + live tracker columns	9 used columns
`<sheet> <year> (Closed)`	An exact frozen snapshot of a tracker sheet at the moment its year was rolled over; one per section per closed year, until purged	mirrors source
`PMS Records`	Every Service Desk maintenance record ever submitted, plus synthetic system-audit rows (rollover/purge events)	70 columns
`PMS Records - Infra & Security`	Every Infrastructure & Security maintenance record ever submitted, including evidence metadata	62 columns
`PMS Tickets`	One row per findings ticket, current state only	22 columns
`PMS Ticket Log`	One append-only row per change to any ticket, forever	9 columns
`PMS Users`	One row per registered technician/administrator; the access roster itself	15 columns
`PMS Notification Recipients`	Opt-in email distribution list for completion and ticket-event notifications	7 columns

Each record/ticket schema is defined once, as an ordered array of `{key, label}` pairs, in its owning module — the array **is** the schema; a header-verification pass compares live sheet headers against it on every access.

7.1 Identifiers

ID type	Format	Example
Maintenance record	`PMS-<year>-T<cycle>-<zero-padded sequence>`	`PMS-2026-T2-014`
Legacy-seed record	`LEGACY-SEED-<year>-<section>-T<cycle>-<32-char hash>`	deterministic from asset+section+cycle, so the same backfill target always resolves to the same row
Findings ticket	`YDC-PMS-<year>-<n>` (unpadded)	`YDC-PMS-2026-7`
Ticket log entry	`<ticketId>-L<2-digit sequence>`	`YDC-PMS-2026-7-L03`

7.2 Computed vs. authoritative roster fields

`PMS Users` stores `email`, `name`, `section`, `active`, `canManageAssets`, and various timestamps as genuinely authoritative, writable fields. `role` and (the sheet's own "Administrator" display column) are **display mirrors only** — recomputed on every read from `isAdmin(email)` (the configuration allowlist check) and `canManageAssets`, never read back as input. A raw sheet edit that types `ASSET_MANAGER` directly into the Role column is honored as a second, human-friendly way to set the same underlying `canManageAssets` boolean; typing `ADMIN` into that column has no effect, since administrator status is never sourced from this sheet at all.

7.3 The completion-state text convention

Rather than a separate status column, the final column of both record sheets (`PMS Completion`) encodes the whole completion decision as one human-readable string: a ten-block Unicode progress bar, a percentage, the raw completed/applicable fraction, an em dash, and a state word (`INCOMPLETE`, `SYNCING`, `COMPLETED`, `SYNC REQUIRED`, `SYNC FAILED`, `HISTORICAL_COMPLETED`). A single shared parser (`PMS.Util.completionState`) extracts the state word by taking everything after the *last* em dash in the string, and this parsed state is the one source of truth every other feature checks — whether a record is locked for editing, whether it counts toward compliance, whether evidence can still be replaced, all key off this same parse rather than a second, potentially-inconsistent flag.

7.4 Secrets and configuration

Nothing sensitive is stored in `PmsConfig.js` (which is source code, visible to anyone with project access) beyond identifiers that are not secrets in themselves (spreadsheet ID, Drive folder IDs). Genuine secrets — the evidence-descriptor HMAC signing key, the additional-administrators allowlist, various operation-state and idempotency-cache markers — live in `PropertiesService`'s Script Properties, generated lazily on first use where applicable (the signing secret is created once, under a lock, the first time it's needed, and never regenerated).

8. Key mechanisms

8.1 Idempotency and duplicate-draft prevention

Every questionnaire submission carries a client-generated idempotency key. `save()` first tries to match an existing row by exact record ID, then by idempotency key; if neither matches — the common signature of a page reload or a retried failed request — it falls back to a natural-key lookup (section, asset tag, cycle, technician) for any open (non-completed) draft and adopts that row instead of creating a new one. A completed record is deliberately excluded from this adoption path, so a genuine second attempt after completion is correctly treated as a new reinspection rather than silently merged into the finished row.

8.2 The permanent completion lock

Once `PMS.Util.completionState(record.pmsCompletion) === 'COMPLETED'`, three independent layers enforce that it can never be edited again: `save()` refuses any further write and returns an idempotent-looking success (never a hard error, so a flaky-connection retry doesn't look like a failure) while appending a best-effort audit event recording the attempt; `PmsEvidence.js` and `PmsRecords.js`'s evidence-attach path both re-check the same state before accepting a replacement file; and the client itself marks the record `readOnly` and disables every input and non-navigation button the moment a completed record is loaded, for its own technician as much as anyone else.

8.3 Evidence integrity

Infrastructure evidence is validated at upload (size, extension/MIME allowlist, and byte-level content sniffing that catches disguised executables/scripts regardless of declared type), then described by a signed, tamper-evident descriptor bound to the specific record, asset, technician, and evidence category via HMAC-SHA256 with constant-time comparison. That descriptor is re-verified — and the live Drive file re-opened, re-hashed, and re-checked for its folder location and trashed state — both immediately after upload and again at the moment a record is finalized as complete, so a file that was moved, altered, or deleted between those two points blocks completion rather than silently finalizing against stale evidence.

8.4 Repair tracking and the ticket-to-tracker mapping

A finding's ticket status is the single source of truth for what a completed asset's tracker cell shows once maintenance itself is done. `PMS.Tracker`'s completion-sync function calls into `PMS.Tickets` to compute the correct cell value from current ticket state every time it writes, rather than the ticket module pushing a value to the tracker only at the moment of a status change — this makes the cell self-healing: any future recomputation (a later ticket change, a reconciliation retry) always derives the cell fresh from ticket state rather than trusting whatever was last written.

8.5 Administrator data-lifecycle operations

Rollover, Year Purge, and Reset (see PRD §9B) each follow the same shape: a read-only preview/dry-run step that issues a short-lived, single-use, requester-bound token; a client-side confirmation phrase (server-validated for Year Purge and Reset, client-only for Rollover); and an execute step, under the script lock, that re-validates everything the preview claimed before writing anything. Year Purge additionally chains a mandatory backup-download step between preview and execute — the server has no code path to reach execute without a real backup token, which is itself only issued after consuming a real preview token, so "delete without a backup" isn't a UI omission that could be bypassed, it's absent from the server entirely.

8.6 Background and scheduled work

Two distinct trigger patterns are used:

- **Recurring daily trigger** (`PMS_sendCycleReminder_`, installed by `PMS.Notify.ensureReminderTrigger`) — a standard Apps Script time-based trigger firing once a day at a configured hour in

`Asia/Manila`. Installation is idempotent: any existing trigger with the same handler name is deleted before a new one is created, so re-running setup can never stack up duplicate daily sends.

- **Self-rescheduling one-shot trigger** (`PMS_continueReconciliation_`) — used when a rollover's post-open reconciliation has more pending records than fit in one execution. Each run processes a bounded batch (50 records), persists a cursor to Script Properties, and — if work remains — schedules itself to fire again roughly a minute later, repeating unattended until the queue drains or a batch reports it needs manual attention.

8.7 Caching

Read-heavy, rarely-changing data (asset lists, ticket lists, dashboard aggregates) is cached via `CacheService`, keyed by a generation counter rather than a fixed TTL for the data that matters most: any write that could invalidate a cached list bumps its generation counter, which changes the cache key on the very next read — so a write is visible immediately to every other viewer rather than waiting out a cache window, while an unrelated read still benefits from the cache between writes. Larger payloads are chunked across multiple cache entries to stay under Apps Script's per-key size limit.

9. In-app administrative screens

Two dashboard screens exist specifically so day-to-day roster and asset-master maintenance never requires opening the raw spreadsheet:

- **Manage Assets** (`PMS.AssetAdmin`) — list is available to any registered user for their own section (administrators can switch sections); create/update/bulk-CSV-update require the Asset Manager grant or administrator status. A bulk update is matched by exact asset tag against the current sheet contents; nothing already present is ever silently removed by an upload.
- **Manage Users** (functions in `PmsAuth.js`, screen described in PRD §11.4) — administrator-only. A single combined update call changes only the fields actually present in the request, deliberately unlike an earlier, narrower function it replaces for this purpose (`adminSetUserSection`) which used to unconditionally rewrite the active flag to `true` on every call — a latent bug (a section edit could silently reactivate a deactivated account) that was fixed as part of building this screen, once the new combined path made the bug reachable in ordinary use rather than only via direct API misuse.

10. Notifications architecture

All outbound mail funnels through `PmsNotify.js`'s shared `send()` / `wrapBody()` helpers, so every notification type shares the same HTML/plain-text templating, the same daily-quota check, and the same "never throw back to the caller" guarantee. Two different recipient-sourcing strategies are used, deliberately:

- **Event-driven notifications** (completion, ticket events) read from the separately curated `PMS Notification Recipients` sheet, filtered by an opt-in checkbox per notification family and an optional section scope. This is appropriate for "I'd like to be copied on this," a genuinely optional subscription.

- **The cycle-deadline reminder** reads directly from the live `PMS Users` roster instead — every active, registered account, with no separate opt-in — because its entire purpose is to reach everyone who still has outstanding PMS work, and a second, manually curated list would risk silently excluding a real technician simply because nobody remembered to add them to it.

A master configuration switch can disable all outbound mail without touching either recipient source.

11. Testing strategy

There is no live Apps Script sandbox suitable for fast, repeatable automated testing, so this project uses a hand-built Node.js test harness that is not part of the shipped application:

- **Stubs** (`stub.js`) implement exactly the subset of `SpreadsheetApp` , `PropertiesService` , `LockService` , `CacheService` , `Session` , `ScriptApp` , `DriveApp` , `MailApp` , `Utilities` , and `HtmlService` behavior this codebase actually exercises — in-memory, deterministic, and fast — including a functioning `deleteTrigger` / `newTrigger` chain so trigger-installation logic can be tested for real, not mocked away.
- **Harness** (`buildHarness()`) loads every real file under `WebApp/*.js` , in the same alphabetical order Apps Script itself uses, into a fresh Node `vm` context per test, against those stubs — meaning the tests exercise the actual production source, not a parallel reimplementation of it.
- **Test file** (`t.js`) is a flat suite of individually named tests, each building its own harness and fixtures from scratch, covering authorization boundaries, validation edge cases, the completion pipeline, duplicate-draft prevention, the ticket-to-tracker mapping, evidence verification, every administrator data-lifecycle tool, and the notification system, including date-dependent logic written to remain correct regardless of which real calendar day the suite happens to run on.
- **Preview servers** (`preview-*.js`) serve the real HTML partials over plain HTTP with a mock `google.script.run` bridging into the same stub environment, letting a real (headless) browser exercise a fully wired UI against realistic seeded data — used throughout development to verify visual and interaction correctness (mobile layout, focus traps, permission-gated controls) that a server-side unit test cannot see.

The standard verification workflow for a change is: implement with a new or updated test; temporarily revert only the production file(s) via `git stash` , confirm the new test actually fails (and fails for the expected reason) against the old code, then restore; apply the change as a `git format-patch` file and verify it applies cleanly and produces a byte-identical tree in a disposable `git worktree` ; then push and deploy.

12. Known constraints

- **Six-minute execution limit.** Any single Apps Script invocation, including a trigger, is capped at six minutes. Every bulk operation (legacy import, rollover reconciliation) is therefore chunked with a persisted cursor and either client-driven continuation or a self-rescheduling trigger, rather than assuming a batch will finish in one call.

- **No true transactions.** `LockService` provides mutual exclusion, not atomicity across multiple sheet writes; multi-step writes (remarks, then status, then final completion state) are deliberately ordered so that a mid-sequence failure leaves the record in a detectable, recoverable state (`SYNC FAILED`) rather than a silently inconsistent one.
- `google.script.run` **serialization.** The structured serializer's silent- `null` -on-large-payload behavior (§5.3) is a real, previously-hit failure mode, not a hypothetical one; every API response is therefore a JSON string, not a returned object.
- **10 million cell ceiling.** Google Sheets' hard limit is the ultimate bound on how much history the workbook can hold; Year Purge exists specifically to let an administrator manage that ceiling deliberately, with a backup, rather than the app degrading unexpectedly as the workbook fills.
- **Single spreadsheet, single points of contention.** `LockService.getScriptLock()` is process-wide for the whole project; a long-held lock (a large legacy-import chunk, a rollover) will make an unrelated concurrent write wait. Lock timeouts are kept short (typically 30 seconds) so a contended operation fails fast with a clear `BUSY` message rather than hanging a user's request.